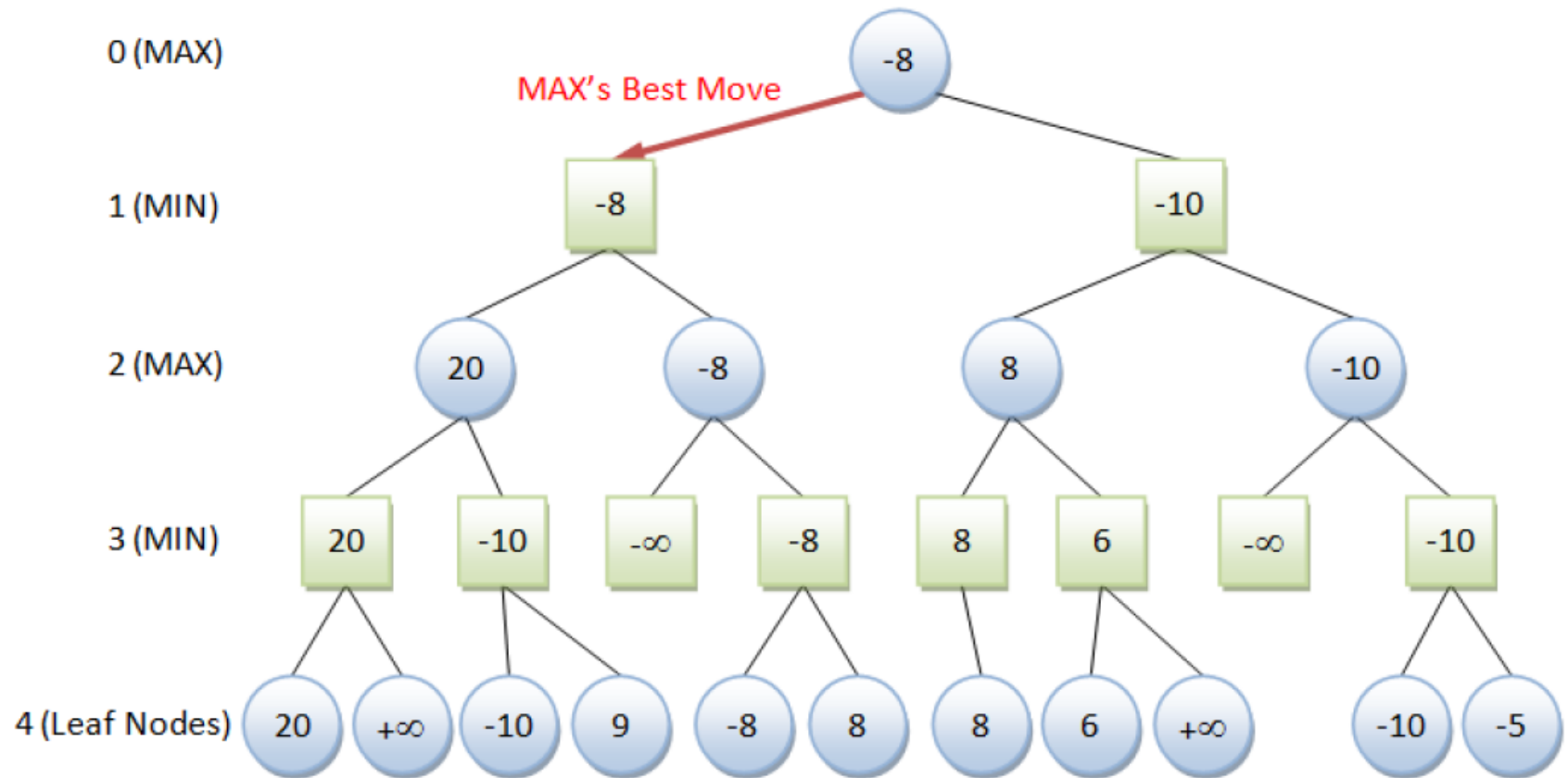


人工智慧導論

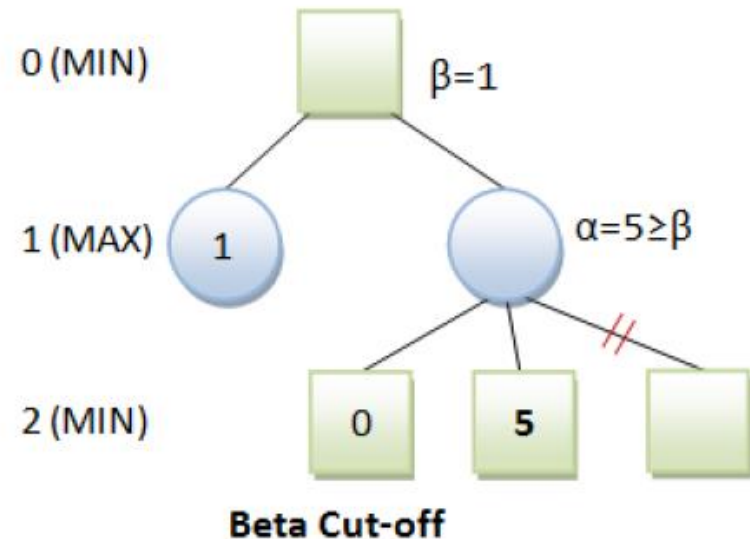
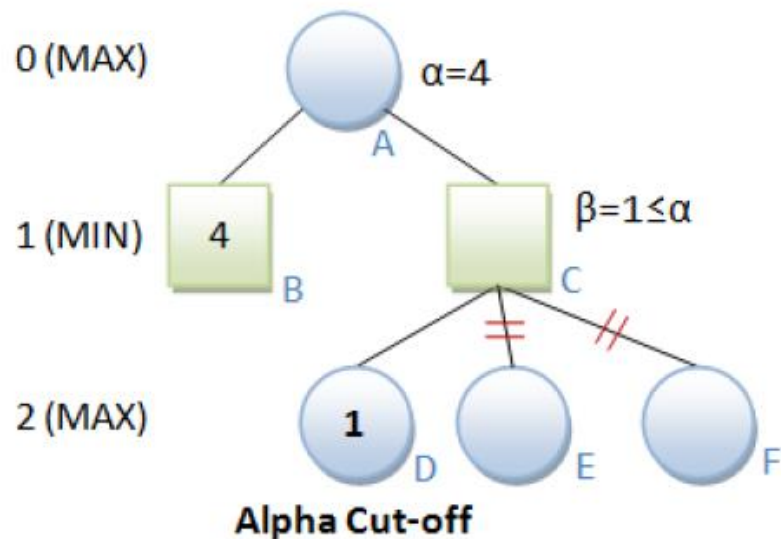


MiniMax

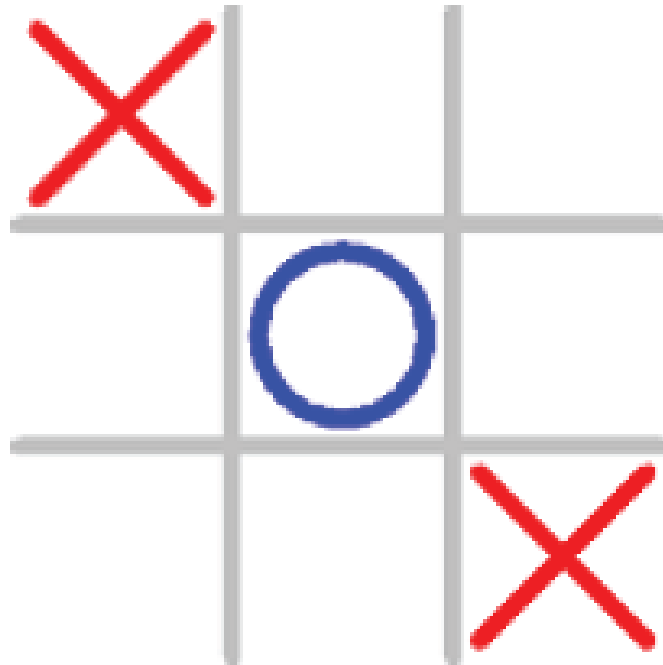
MiniMax Search Algorithm



Alpha-Beta Pruning



Tic-Tac-Toe



Evaluation Function

A better evaluation function for Tic-Tac-Toe is:

- +100 for EACH 3-in-a-line for computer.
- +10 for EACH 2-in-a-line (with a empty cell) for computer.
- +1 for EACH 1-in-a-line (with two empty cells) for computer.
- Negative scores for opponent, i.e., -100, -10, -1 for EACH opponent's 3-in-a-line, 2-in-a-line and 1-in-a-line.
- 0 otherwise (empty lines or lines with both computer's and opponent's seed).

Compute the scores for each of the 8 lines (3 rows, 3 columns and 2 diagonals) and obtain the sum.

The principle of minimax is to *minimize the maximum possible loss*.

MiniMax Algorithm

```
minimax(level, player) // player may be "computer" or "opponent"
if (gameover || level == 0)
    return score
children = all legal moves for this player
if (player is computer, i.e., maximizing player)
    // find max
    bestScore = -inf
    for each child
        score = minimax(level - 1, opponent)
        if (score > bestScore) bestScore = score
    return bestScore
else (player is opponent, i.e., minimizing player)
    // find min
    bestScore = +inf
    for each child
        score = minimax(level - 1, computer)
        if (score < bestScore) bestScore = score
    return bestScore

// Initial call
minimax(2, computer)
```

Alpha-Beta Algorithm

```
minimax(level, player, alpha, beta) // player may be "computer" or "opponent"
if (gameover || level == 0)
    return score
children = all valid moves for this "player"
if (player is computer, i.e., max's turn)
    // Find max and store in alpha
    for each child
        score = minimax(level - 1, opponent, alpha, beta)
        if (score > alpha) alpha = score
        if (alpha >= beta) break; // beta cut-off
    return alpha
else (player is opponent, i.e., min's turn)
    // Find min and store in beta
    for each child
        score = minimax(level - 1, computer, alpha, beta)
        if (score < beta) beta = score
        if (alpha >= beta) break; // alpha cut-off
    return beta

// Initial call with alpha=-inf and beta=inf
minimax(2, computer, -inf, +inf)
```